N.Bharath Kumar
nersubharath@gmail.com

Task Module: Day 6 Authentication & CRUD using JWT

Project Environment & Architecture Setup

The codebase relies on a modular separation of concerns strategy. Before code implementation, structural development dependencies and primary packages were added to the core configuration workspace environment.

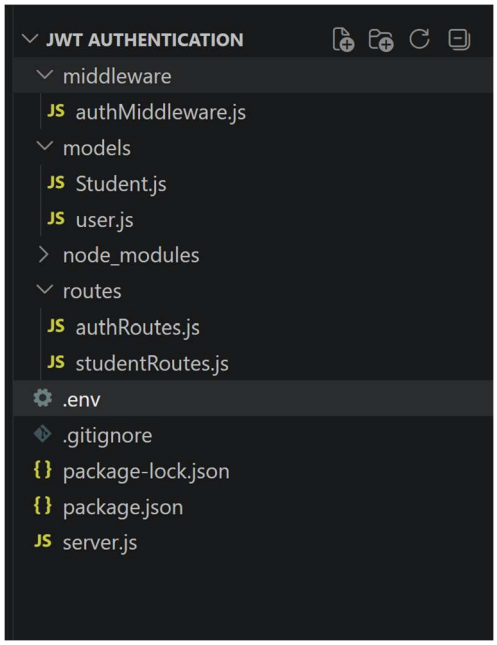
Dependency Manifest & Justifications

Package / Dependency	Functional Classification	Operational Architecture Importance
express	Core Framework	Handles HTTP pipeline routing and response structures cleanly.
mongoose	Database Layer	Forces structured schema data validations onto flexible MongoDB documents.
dotenv	Security Config	Prevents high-risk vulnerability exposures by isolating system security secrets.
bcryptjs	Cryptography Engine	Executes secure standard text key stretching algorithms with 10 salt rounds.
jsonwebtoken	Session Authorization	Issues encrypted cryptographically verifiable state authentication tokens.
cors	Network Firewall Configuration	Manages cross-origin resource access safety policies for frontend interactions.
nodemon	Development Environment Utility	Monitors codebase delta adjustments and applies clean hot-reloads.

N.Bharath Kumar
nersubharath@gmail.com

Project Folder Tree Architecture

The workspace structure mirrors strict separation principles to streamline maintenance:



Task 1: User Authentication System

Objective: Build registration and login handling systems incorporating structural cryptography standards to protect identity data records.

Source Implementation:models/User.js

```
const mongoose = require('mongoose');

// Why a Schema? It guarantees every user object has these exact fields before hitting
const userSchema = new mongoose.Schema({
    name: {
        type: String,
        required: [true, "Name is required"] // Task 3: Built-in validation prevents er
    },
    email: {
        type: String,
        required: [true, "Email is required"],
        unique: true, // Prevents duplicate registrations
        lowercase: true
    },
    password: {
        type: String,
        required: [true, "Password is required"],
        minlength: [6, "Password must be at least 6 characters long"]
    }
}, { timestamps: true }); // Automatically tracks 'createdAt' and 'updatedAt' fields

module.exports = mongoose.model('User', userSchema);
```

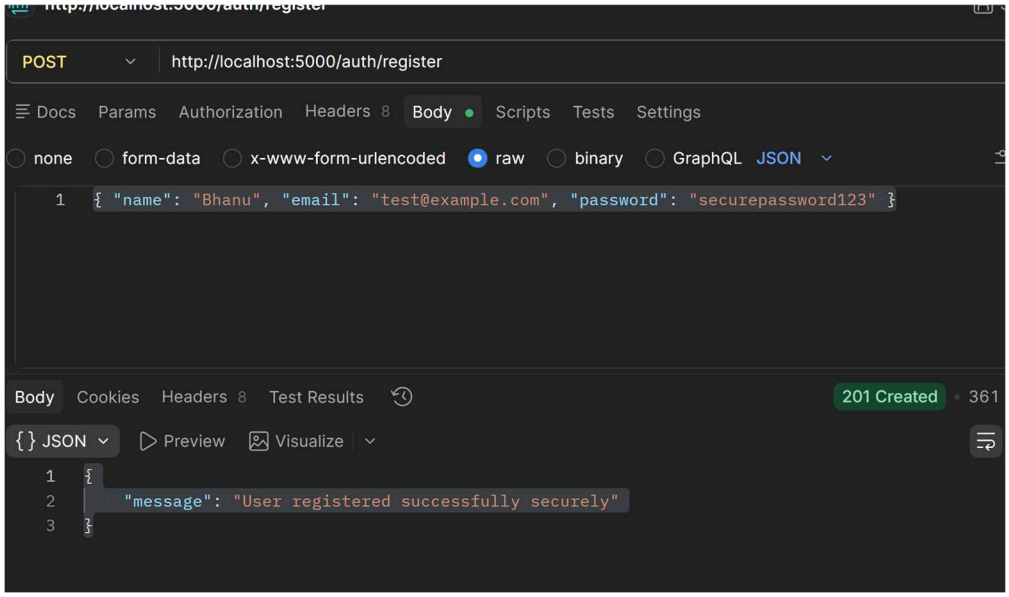
N.Bharath Kumar
nersubharath@gmail.com

Source Implementation: routes/authRoutes.js

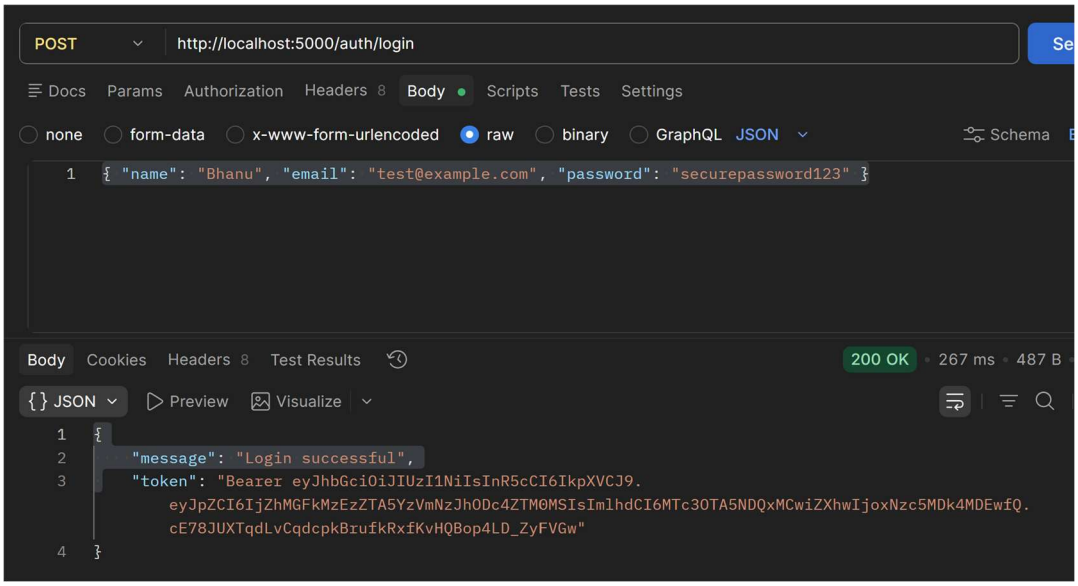
```
const express = require('express');
const router = express.Router();
const User = require('../models/User');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

// ========================================
// TASK 1: REGISTER API
// ========================================
router.post('/register', async (req, res) => {
    try {
        const { name, email, password } = req.body;

        // Task 3 Validation: Check for empty submissions explicitly
        if (!name || !email || !password) {
            return res.status(400).json({ message: "All fields are required" });
        }

        // Check if user already exists
        const userExists = await User.findOne({ email });
        if (userExists) {
            return res.status(400).json({ message: "Email already registered" });
        }

        // 10 is the industry sweet spot between speed and un-crackable security.
        const salt = await bcrypt.genSalt(10);
        const hashedPassword = await bcrypt.hash(password, salt);

        // Save to MongoDB
        const newUser = new User({
            name,
            email,
            password: hashedPassword // Save the secure hash, never the raw password!
        });

        await newUser.save();
        res.status(201).json({ message: "User registered successfully securely" });

    } catch (error) {
        // Task 3: Proper error handling
        res.status(500).json({ message: "Server error", error: error.message });
    }
});

```

N.Bharath Kumar
nersubharath@gmail.com

```
// TASK 1: LOGIN API
// ========================================
router.post('/login', async (req, res) => {
    try {
        const { email, password } = req.body;

        if (!email || !password) {
            return res.status(400).json({ message: "Please provide email and password" });
        }

        // Look for the user
        const user = await User.findOne({ email });
        if (!user) {
            return res.status(400).json({ message: "Invalid credentials" }); // Don't explici
        }

        // Compare entered password with the hashed password in DB
        const isMatch = await bcrypt.compare(password, user.password);
        if (!isMatch) {
            return res.status(400).json({ message: "Invalid credentials" });
        }

        // Generate JWT Token
        // Why pass user._id? This payload travels inside the token, telling the server later
        const token = jwt.sign(
            { id: user._id },
            process.env.JWT_SECRET,
            { expiresIn: '1h' } // Token expires in 1 hour for safety
        );

        res.status(200).json({
            message: "Login successful",
            token: `Bearer ${token}` // Standard formatting prefix
        });

    } catch (error) {
        res.status(500).json({ message: "Server error", error: error.message });
    }
});

module.exports = router;
```



N.Bharath Kumar
nersubharath@gmail.com



Postman Output showing POST /auth/login request returning 'Bearer eyJ...' Token string

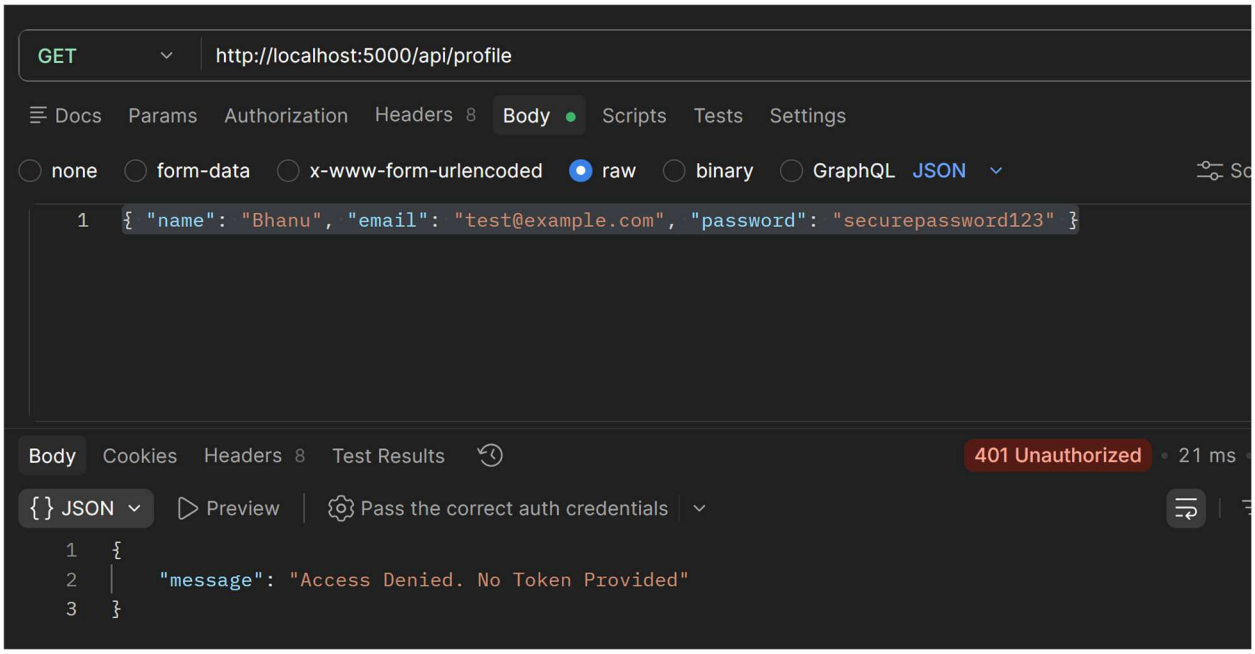
---

Task 2: Protected Route Implementation

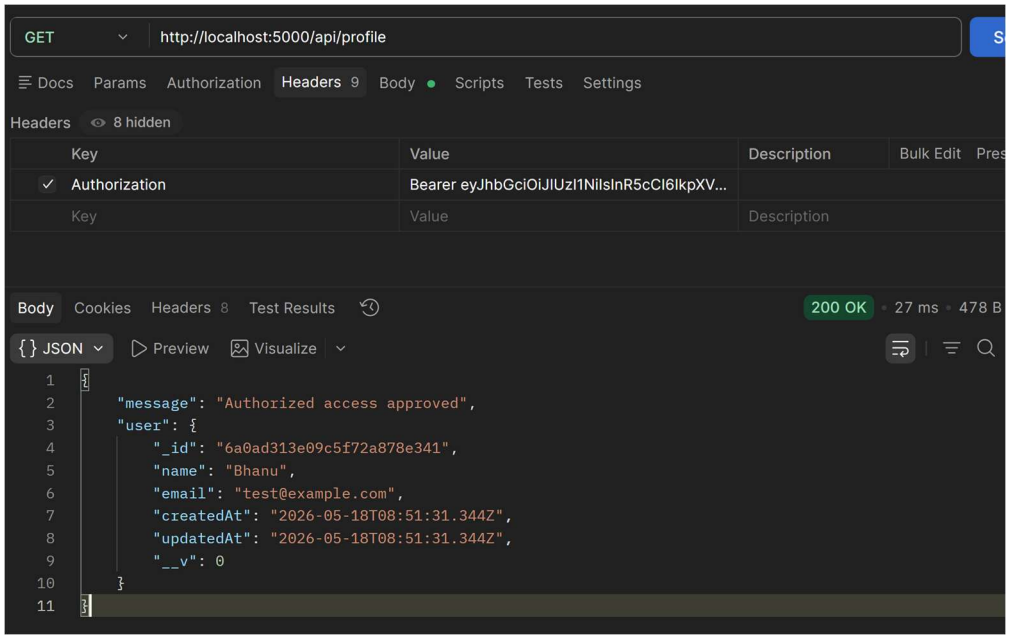
Objective: Build a custom intercepting middleware system to parse network transmission packets and protect specialized system scopes.**Source Implementation: middleware/authMiddleware.js**

```
middleware > JS authMiddleware.js > ...
1   const jwt = require('jsonwebtoken');
2
3   const authMiddleware = (req, res, next) => {
4       try {
5           // Get token from headers
6           const authHeader = req.header("Authorization");
7
8           // Task 2: Return proper error messages for missing tokens
9           if (!authHeader || !authHeader.startsWith("Bearer ")) {
10              return res.status(401).json({ message: "Access Denied. No Token Provided" });
11          }
12
13          // Extract the raw token string by splitting the space between "Bearer" and the token
14          const token = authHeader.split(" ")[1];
15
16          // Verify token against our secret key
17          const verified = jwt.verify(token, process.env.JWT_SECRET);
18
19          // Attach the verified payload (the user ID) to the request object so subsequent endpoints can read it
20          req.user = verified;
21
22          // next() tells Express to move past this gatekeeper to the actual API logic
23          next();
24
25      } catch (error) {
26          // Task 2: Return proper error messages for invalid tokens
27          res.status(401).json({ message: "Invalid or expired Token" });
28      }
29  };
30
31  module.exports = authMiddleware;
```

N.Bharath Kumar
nersubharath@gmail.com



Postman GET /api/profile rejection trace with Status 401 when missing an Authorization header]



Postman GET /api/profile success trace with Status 200 when sending valid Bearer token credentials]

N.Bharath Kumar
nersubharath@gmail.com

Task 3: CRUD API Enhancement

Objective: Refactor data manipulation paths to support field validations, database error catches, and strict schema compliance rules.

Source Implementation: models/Student.js

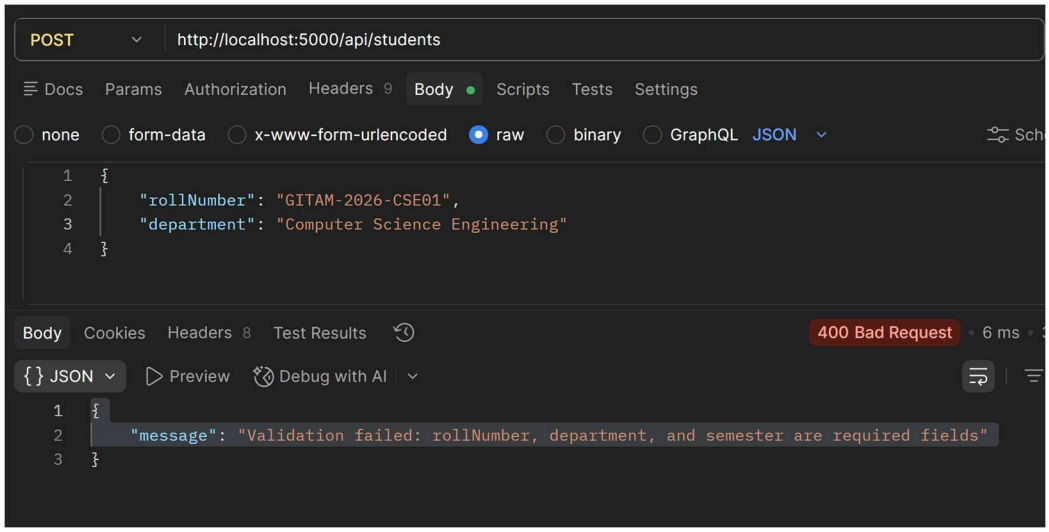
```
const mongoose = require('mongoose');

const studentSchema = new mongoose.Schema({
    // LINK TO USER: This links the student profile to their login account
    userId: {
        type: mongoose.Schema.Types.ObjectId,
        ref: 'User',
        required: true
    },
    rollNumber: {
        type: String,
        required: [true, "Roll number is required"],
        unique: true
    },
    department: {
        type: String,
        required: [true, "Department is required"],
        trim: true // Task 3: Cleans up accidental trailing spaces
    },
    semester: {
        type: Number,
        required: [true, "Semester is required"],
        min: [1, "Semester cannot be less than 1"],
        max: [8, "Semester cannot be more than 8"] // Built-in data validation
    }
}, { timestamps: true });

module.exports = mongoose.model('Student', studentSchema);
```

Source Implementation Snippet: routes/studentRoutes.js (POST Validation Flow)

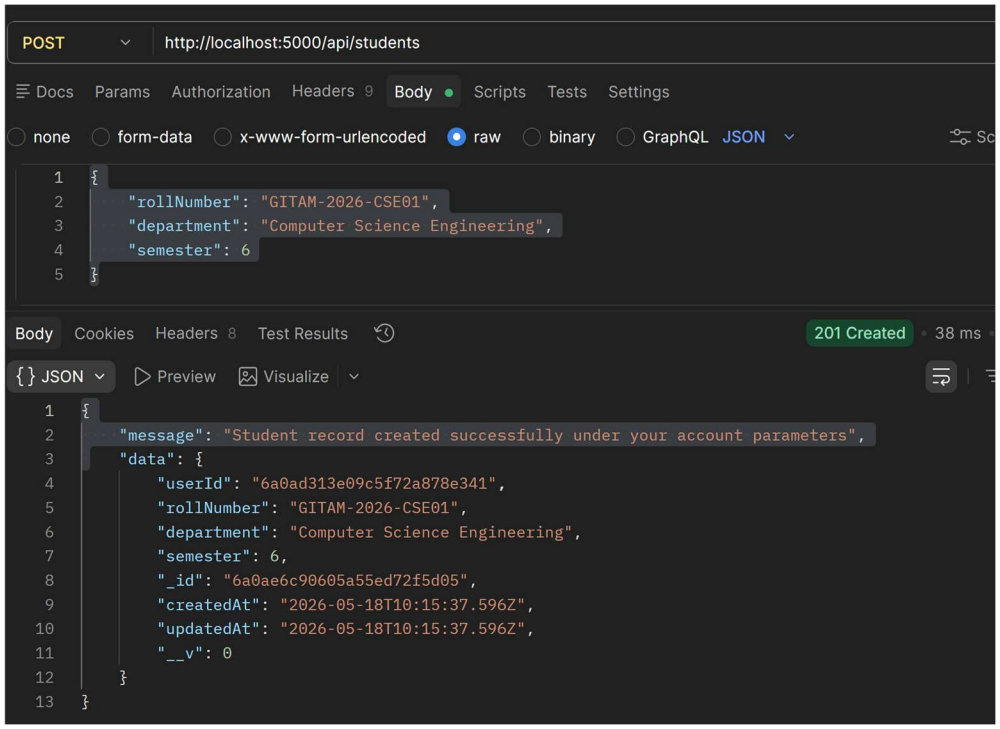
```
const express = require('express');
const router = express.Router();
const authMiddleware = require('../middleware/authMiddleware');
const User = require('../models/User');

// Task 2: GET /profile protected route
// Note how authMiddleware sits in the middle before the async logic executes!
router.get('/profile', authMiddleware, async (req, res) => {
    try {
        // req.user was set by our middleware if verification succeeded
        const user = await User.findById(req.user.id).select('-password'); // '-password' explicitly excludes the hash from returning
        res.status(200).json({ message: "Authorized access approved", user });
    } catch (error) {
        res.status(500).json({ message: "Server error", error: error.message });
    }
});

module.exports = router;
```

N.Bharath Kumar
nersubharath@gmail.com

[INSERT SCREENSHOT: Postman Trace showing a validation block rejection (400 Bad Request) for an empty payload field]



[INSERT SCREENSHOT: Postman Trace showing successful CRUD creation (201 Created) under valid authenticated parameters]



N.Bharath Kumar
nersubharath@gmail.com

Task 4 & Environment Verifications

MongoDB Local Infrastructure Verification

The connection pipeline binds directly to the localized system instance. System operation trace confirmations:

[INSERT SCREENSHOT: Terminal window showing 'MongoDB connected successfully to local instance' console initialization output]